



SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Information Science & Engineering

Continuous Internal Evaluation Test III - ODD Semester 2025 - 26

Scheme and Solutions of Evaluation

Course Title: **Software Testing**

Course Code: **IS722T2I**

Prepared By: **Mrs. Aishwarya M Bhat / Mrs. Shwetha S Shetty**

Semester: **VII**

Q. No.		Solution	Marks Distribution
1.	a.	In any test automation scenario, the System Under Test (SUT) is the application or component being tested. SUT interaction technologies are the mechanisms through which automated tests communicate with the SUT. They determine how the test script observes, manipulates, and verifies behavior. Whether you write the tests in code or another tool, the test must interact with the SUT somehow in order to test it. The most obvious way to do that is by simulating the user's interaction with the UI. But this is not always the best option. 1. GUI-Based Interaction Automation tools interact with the user interface the same way a human does. Examples: • Selenium WebDriver for web pages • Appium for mobile apps • Playwright and Cypress • Desktop automation (WinAppDriver, Robot Framework + AutoIt) How It Works: • Locators (XPath, CSS selectors, accessibility IDs) are used to find UI elements. • Actions (click, type, scroll) simulate user behavior. Importance: • Validates actual user flows • Technology-agnostic (browser-based testing works regardless of backend) • Detects UI bugs early 2. API-Level Interaction Testing is done by calling public interfaces, bypassing the UI. Examples: • REST/GraphQL tests using Postman, RestAssured, Playwright API, or Python requests How It Works:	2.0 3.0 2.0

		Tests send: • HTTP requests • JSON payloads • authentication headers Then validate: • status codes • response bodies • performance (latency, throughput) Importance: • Fast, stable, maintainable • Allows pinpointing backend issues separately from the UI • Essential for microservices and distributed systems	3.0																					
2.	a.	Differences between Unit Testing frameworks and BDD-Style frameworks are as follows. <table><tr><th>Aspect</th><th>Unit Testing Frameworks</th><th>BDD-Style Frameworks</th></tr><tr><td>Purpose</td><td>Verify code correctness at small scale</td><td>Verify system behavior at user/business level</td></tr><tr><td>Audience</td><td>Mainly developers</td><td>Developers + QA + business stakeholders</td></tr><tr><td>Typical Scope</td><td>Functions, classes, modules</td><td>Workflows, user stories, acceptance criteria</td></tr><tr><td>Structure</td><td>Setup → call function → assert</td><td>Given → When → Then</td></tr><tr><td>Toolchain</td><td>xUnit, unittest, pytest</td><td>Cucumber, SpecFlow, Jasmine, Behave</td></tr><tr><td>Dependency Management</td><td>Heavy use of mocks</td><td>Typically, fewer mocks; tests run through the system</td></tr></table> Use Unit Testing Frameworks when: • You need fast feedback during development • Testing library functions, pure logic, microservices internals • You want high test coverage with low cost • You need deterministic, isolated tests Use BDD Frameworks when: • Requirements must be readable by non-developers • You need tests that double as documentation • You are automating user journeys or acceptance criteria • You want a clear translation from user stories → tests	Aspect	Unit Testing Frameworks	BDD-Style Frameworks	Purpose	Verify code correctness at small scale	Verify system behavior at user/business level	Audience	Mainly developers	Developers + QA + business stakeholders	Typical Scope	Functions, classes, modules	Workflows, user stories, acceptance criteria	Structure	Setup → call function → assert	Given → When → Then	Toolchain	xUnit, unittest, pytest	Cucumber, SpecFlow, Jasmine, Behave	Dependency Management	Heavy use of mocks	Typically, fewer mocks; tests run through the system	1*6=6.0 2.0 2.0
Aspect	Unit Testing Frameworks	BDD-Style Frameworks																						
Purpose	Verify code correctness at small scale	Verify system behavior at user/business level																						
Audience	Mainly developers	Developers + QA + business stakeholders																						
Typical Scope	Functions, classes, modules	Workflows, user stories, acceptance criteria																						
Structure	Setup → call function → assert	Given → When → Then																						
Toolchain	xUnit, unittest, pytest	Cucumber, SpecFlow, Jasmine, Behave																						
Dependency Management	Heavy use of mocks	Typically, fewer mocks; tests run through the system																						
3.	a.	1. Class Name • Locates elements by the value of their class attribute. • Example: <code>driver.findElement(By.className("btn"));</code> 2. CSS Selector	1.0																					

	• Uses CSS selectors to locate elements. • Example: <code>driver.findElement(By.cssSelector("div.container > p"));</code>	1.0
	3. ID • Locates elements by their id attribute, which is often unique. • Example: <code>driver.findElement(By.id("change-password"));</code>	1.0
	4. Link Text • Locates a link by the exact text displayed. • Example: <code>driver.findElement(By.linkText("Forgotten Password"));</code>	1.0
	5. Partial Link Text • Locates a link by a partial match of its text. • Example: <code>driver.findElement(By.partialLinkText("Forgotten"));</code>	1.0
	6. Name • Locates elements by the name attribute, typically used for form inputs. • Example: <code>driver.findElement(By.name("email"));</code>	1.0
	7. Tag Name • Locates elements by their HTML tag name. • Example: <code>driver.findElement(By.tagName("h1"));</code>	2.0
	8. XPath • Uses XPath expressions to locate elements, providing flexibility for complex structures. • Example: <code>driver.findElement(By.xpath("//input[@name='email']"));</code>	2.0
b.	<pre>import time from selenium import webdriver from selenium.webdriver.common.by import By class GoogleMapsAutomation: def setUp(self): # Initialize the Chrome browser self.driver = webdriver.Chrome() # Ensure ChromeDriver is in your PATH def search_location(self): try: g_link = "https://www.google.com/maps" ip_link = '//*[@id="searchboxinput"]' directions_xpath = '//*[@id="searchbox-searchbutton"]' # Adjusted for the correct XPath for the search button # Open Google Maps self.driver.get(g_link) time.sleep(2) # Input location from user name = str(input("Enter Location: ")) search_input = self.driver.find_element(By.XPATH, ip_link) search_input.send_keys(name)</pre>	1.0

4.

a.

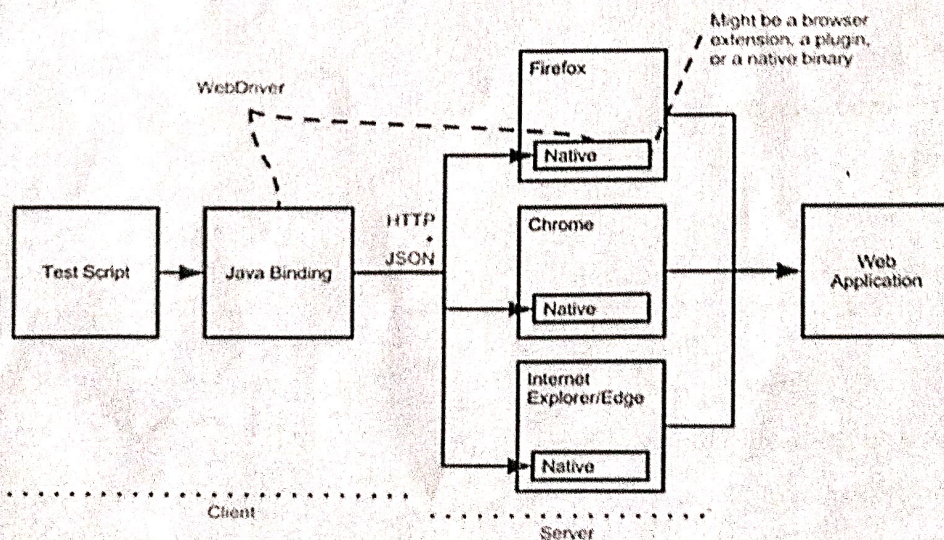
Selenium WebDriver is a powerful tool used for automating web browsers by simulating user interactions such as opening a website, clicking on links, filling out forms, and navigating through pages.

Benefits of Using WebDriver:

- **Multi-browser support:** WebDriver works with all major browsers like Chrome, Firefox, Safari, and more, providing flexibility in testing across different platforms.
- **Supports multiple programming languages:** You can write your test scripts in various programming languages such as Java, Python, C#, Ruby, and others.
- **Mimics real user actions:** WebDriver performs browser actions using native events, making interactions more realistic compared to older Selenium versions.
- **Open-source:** Selenium WebDriver is free to use and has strong support from a vast community of developers.

Working-

- WebDriver is a powerful tool that works with all major web browsers and supports various programming languages for automated testing.
- It achieves this through a combination of components: the web browser itself, a plugin or extension within the browser that implements the WebDriver JSON API, and language bindings (e.g., Java) that make HTTP requests to this API.
- When a test script runs, it opens the browser and activates the plugin, allowing the script to send requests to perform actions like clicking links or typing text.
- The plugin acts as a server, receiving commands and translating them into browser actions.
- Since the plugin only needs to implement the JSON API, it can be created for any major browser, ensuring compatibility.
- This structure also means that WebDriver's language bindings can communicate with the browser regardless of the programming language being used, making it a flexible solution for automated web testing.



Web Driver working principle

b.	<pre> from selenium import webdriver from selenium.webdriver.common.by import By import time class GoogleFormAutomation: def setUp(self): self.driver = webdriver.Chrome() def fill_form(self): self.driver.get("https://forms.gle/vDCCNyhkBhBsUCja9") time.sleep(2) name = "Your Name" name_input = self.driver.find_element(By.XPATH, '//input[@aria-labelledby="i1 i4"]') name_input.send_keys(name) time.sleep(2) usn = "Your USN" age_input = self.driver.find_element(By.XPATH, '//input[@aria-labelledby="i6 i9"]') age_input.send_keys(usn) time.sleep(2) submit=self.driver.find_element(By.XPATH,'//*[@id="mG61Hd"]/div[2]/div/div[3]/div[1]/div[1]]div/span/span') submit.click() time.sleep(2) def tearDown(self): self.driver.quit() if __name__ == "__main__": automation = GoogleFormAutomation() automation.setUp() automation.fill_form() automation.tearDown() </pre>	1.0 1.0 1.0 1.0 1.0
c.	<pre> import unittest from selenium import webdriver from selenium.webdriver.common.by import By class Count(unittest.TestCase): def setUp(self): </pre>	1.0

	<pre> # Set up the WebDriver for Chrome self.driver = webdriver.Chrome() # Make sure to have the ChromeDriver installed self.driver.get("http://www.google.co.in") self.driver.maximize_window() def test_count_elements(self): driver = self.driver # Get all links on the page links = driver.find_elements(By.TAG_NAME, "a") print(f'TOTAL NO OF LINKS = {len(links)}') # Get all buttons on the page buttons = driver.find_elements(By.TAG_NAME, "button") print(f'TOTAL NO OF BUTTONS = {len(buttons)}') # Get all input fields on the page input_fields = driver.find_elements(By.TAG_NAME, "input") print(f'TOTAL NO OF INPUT FIELDS = {len(input_fields)}') def tearDown(self): # Stop the WebDriver self.driver.quit() if __name__ == "__main__": unittest.main() </pre>	1.0 1.0 1.0 1.0
--	---	---

Proceeded
to test case
✓

AR
20/11/25

A. Pawar
20/11/25